



Lecture 9

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

August 11, 2026

Parsing

- Process of determination whether a string can be generated by a grammar.

Parsing

- Process of determination whether a string can be generated by a grammar.
- Parsing falls in two categories:

Parsing

- Process of determination whether a string can be generated by a grammar.
- Parsing falls in two categories:
 - ▶ **Top-down parsing:** Construction of the parse tree starts at the root (from the start symbol) and proceeds towards leaves (token or terminals). Ex ANTLR

Parsing

- Process of determination whether a string can be generated by a grammar.
- Parsing falls in two categories:
 - ▶ **Top-down parsing:** Construction of the parse tree starts at the root (from the start symbol) and proceeds towards leaves (token or terminals). Ex ANTLR (ANother Tool for Language Recognition)

Parsing

- Process of determination whether a string can be generated by a grammar.
- Parsing falls in two categories:
 - ▶ **Top-down parsing:** Construction of the parse tree starts at the root (from the start symbol) and proceeds towards leaves (token or terminals). Ex ANTLR (ANother Tool for Language Recognition)
 - ▶ **Bottom-up parsing:** Construction of the parse tree starts from the leaf nodes (tokens or terminals of the grammar) and proceeds towards root (start symbol). Ex YACC and BISON

Top Down Parsing

- Construction of a parse tree is done by starting the root labeled by a start symbol

Top Down Parsing

- Construction of a parse tree is done by starting the root labeled by a start symbol
- repeat following two steps

Top Down Parsing

- Construction of a parse tree is done by starting the root labeled by a start symbol
- repeat following two steps
 - ▶ at a node labeled with non terminal A select one of the productions of A and construct children nodes (Which production?)

Top Down Parsing

- Construction of a parse tree is done by starting the root labeled by a start symbol
- repeat following two steps
 - ▶ at a node labeled with non terminal A select one of the productions of A and construct children nodes (Which production?)
 - ▶ find the next node at which subtree is Constructed (Which node?)

Top Down Parsing

- Construction of a parse tree is done by starting the root labeled by a start symbol
- repeat following two steps
 - ▶ at a node labeled with non terminal A select one of the productions of A and construct children nodes (Which production?)
 - ▶ find the next node at which subtree is Constructed (Which node?)

Recursive Descent parsing

Algorithm A()

- 1: Choose an A-production, $A \rightarrow X_1 X_2 \cdots X_k$
 - 2: **for** $i = 1$ to k **do**
 - 3: **if** X_i is a nonterminal **then**
 - 4: call procedure $X_i()$
 - 5: **else if** X_i equals the current input symbol α **then**
 - 6: advance the input to the next symbol
 - 7: **else**
 - 8: error()
 - 9: **end if**
 - 10: **end for**
-

Recursive Descent parsing

- Non-deterministic due to line 1 of the Algorithm 1

Recursive Descent parsing

- Non-deterministic due to line 1 of the Algorithm 1
- Require backtracking

Recursive Descent parsing

- Non-deterministic due to line 1 of the Algorithm 1
- Require backtracking
- May require repeated scans over the input.

Recursive Descent parsing

- Non-deterministic due to line 1 of the Algorithm 1
- Require backtracking
- May require repeated scans over the input.
- Dynamic Programming or tabular method may be used.

Left Recursion

- A top-down parser with production $A \rightarrow A\alpha$ may loop forever.

Left Recursion

- A top-down parser with production $A \rightarrow A\alpha$ may loop forever.
- From the grammar $A \rightarrow A\alpha|\beta$ left recursion may be eliminated by transforming the grammar to

Left Recursion

- A top-down parser with production $A \rightarrow A\alpha$ may loop forever.
- From the grammar $A \rightarrow A\alpha|\beta$ left recursion may be eliminated by transforming the grammar to

$$A \rightarrow \beta R$$

Left Recursion

- A top-down parser with production $A \rightarrow A\alpha$ may loop forever.
- From the grammar $A \rightarrow A\alpha|\beta$ left recursion may be eliminated by transforming the grammar to

$$A \rightarrow \beta R$$

$$R \rightarrow \alpha R|\epsilon$$

Left Recursion

- A top-down parser with production $A \rightarrow A\alpha$ may loop forever.
- From the grammar $A \rightarrow A\alpha|\beta$ left recursion may be eliminated by transforming the grammar to

$$A \rightarrow \beta R$$

$$R \rightarrow \alpha R|\epsilon$$

- In general $A \rightarrow A\alpha_1|A\alpha_2|\cdots|A\alpha_m|\beta_1|\beta_2|\cdots|\beta_n$ transforms to

Left Recursion

- A top-down parser with production $A \rightarrow A\alpha$ may loop forever.
- From the grammar $A \rightarrow A\alpha|\beta$ left recursion may be eliminated by transforming the grammar to

$$A \rightarrow \beta R$$

$$R \rightarrow \alpha R|\epsilon$$

- In general $A \rightarrow A\alpha_1|A\alpha_2|\dots|A\alpha_m|\beta_1|\beta_2|\dots|\beta_n$ transforms to

$$A \rightarrow \beta_1 A'|\beta_2 A'|\dots|\beta_n A'$$

$$A' \rightarrow \alpha_1 A'|\alpha_2 A'|\dots|\alpha_m A'|\epsilon$$

Example

- Consider grammar for arithmetic expressions

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Example

- Consider grammar for arithmetic expressions

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- After removal of left recursion the grammar becomes

Example

- Consider grammar for arithmetic expressions

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- After removal of left recursion the grammar becomes

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

Left recursion hidden due to many productions

- Left recursion may also be introduced by two or more grammar rules. For example:

$$S \rightarrow Aa|b$$

$$A \rightarrow Ac|Sd|\epsilon$$

Left recursion hidden due to many productions

- Left recursion may also be introduced by two or more grammar rules. For example:

$$S \rightarrow Aa|b$$

$$A \rightarrow Ac|Sd|\epsilon$$

- Hidden left recursion due to $S \rightarrow Aa \rightarrow Sda$

Left recursion hidden due to many productions

- Left recursion may also be introduced by two or more grammar rules. For example:
 $S \rightarrow Aa|b$
 $A \rightarrow Ac|Sd|\epsilon$
- Hidden left recursion due to $S \rightarrow Aa \rightarrow Sda$
- Remove left recursion systematically.

Left recursion hidden due to many productions

- Left recursion may also be introduced by two or more grammar rules. For example:
 $S \rightarrow Aa|b$
 $A \rightarrow Ac|Sd|\epsilon$
- Hidden left recursion due to $S \rightarrow Aa \rightarrow Sda$
- Remove left recursion systematically.
 - ▶ Starting from the first rule and replacing all the occurrences of the first non terminal symbol.

Left recursion hidden due to many productions

- Left recursion may also be introduced by two or more grammar rules. For example:
 $S \rightarrow Aa|b$
 $A \rightarrow Ac|Sd|\epsilon$
- Hidden left recursion due to $S \rightarrow Aa \rightarrow Sda$
- Remove left recursion systematically.
 - ▶ Starting from the first rule and replacing all the occurrences of the first non terminal symbol.
 - ▶ Removing left recursion from the modified grammar.

Left recursion hidden due to many productions

- Left recursion may also be introduced by two or more grammar rules. For example:
 $S \rightarrow Aa|b$
 $A \rightarrow Ac|Sd|\epsilon$
- Hidden left recursion due to $S \rightarrow Aa \rightarrow Sda$
- Remove left recursion systematically.
 - ▶ Starting from the first rule and replacing all the occurrences of the first non terminal symbol.
 - ▶ Removing left recursion from the modified grammar.
- After the first step (substitute S by its rhs in the rules) the grammar becomes

Left recursion hidden due to many productions

- Left recursion may also be introduced by two or more grammar rules. For example:
 $S \rightarrow Aa|b$
 $A \rightarrow Ac|Sd|\epsilon$
- Hidden left recursion due to $S \rightarrow Aa \rightarrow Sda$
- Remove left recursion systematically.
 - ▶ Starting from the first rule and replacing all the occurrences of the first non terminal symbol.
 - ▶ Removing left recursion from the modified grammar.
- After the first step (substitute S by its rhs in the rules) the grammar becomes
 $S \rightarrow Aa|b$
 $A \rightarrow Ac|Aad|bd|\epsilon$

Left recursion hidden due to many productions

- Left recursion may also be introduced by two or more grammar rules. For example:
 $S \rightarrow Aa|b$
 $A \rightarrow Ac|Sd|\epsilon$
- Hidden left recursion due to $S \rightarrow Aa \rightarrow Sda$
- Remove left recursion systematically.
 - ▶ Starting from the first rule and replacing all the occurrences of the first non terminal symbol.
 - ▶ Removing left recursion from the modified grammar.
- After the first step (substitute S by its rhs in the rules) the grammar becomes
 $S \rightarrow Aa|b$
 $A \rightarrow Ac|Aad|bd|\epsilon$
- After the second step (removal of left recursion) the grammar becomes

Left recursion hidden due to many productions

- Left recursion may also be introduced by two or more grammar rules. For example:
 $S \rightarrow Aa|b$
 $A \rightarrow Ac|Sd|\epsilon$
- Hidden left recursion due to $S \rightarrow Aa \rightarrow Sda$
- Remove left recursion systematically.
 - ▶ Starting from the first rule and replacing all the occurrences of the first non terminal symbol.
 - ▶ Removing left recursion from the modified grammar.
- After the first step (substitute S by its rhs in the rules) the grammar becomes
 $S \rightarrow Aa|b$
 $A \rightarrow Ac|Aad|bd|\epsilon$
- After the second step (removal of left recursion) the grammar becomes
 $S \rightarrow Aa|b$
 $A \rightarrow bdA'|A'$
 $A' \rightarrow cA'|adA'|\epsilon$

Left Factoring

- In top-down parsing when it is not clear which production to choose for expansion of a symbol defer the decision till we have seen enough input.

Left Factoring

- In top-down parsing when it is not clear which production to choose for expansion of a symbol defer the decision till we have seen enough input.
- $A \rightarrow \alpha\beta_1 | \alpha\beta_2$ transforms to

Left Factoring

- In top-down parsing when it is not clear which production to choose for expansion of a symbol defer the decision till we have seen enough input.
- $A \rightarrow \alpha\beta_1 | \alpha\beta_2$ transforms to
- $A \rightarrow \alpha A'$
 $A' \rightarrow \beta_1 | \beta_2$

Predictive Parser

- A non recursive top down parsing method

Predictive Parser

- A non recursive top down parsing method
- Parser “predicts” which production to use

Predictive Parser

- A non recursive top down parsing method
- Parser “predicts” which production to use
- It removes backtracking by fixing one production for every non- terminal and input token(s)

Predictive Parser

- A non recursive top down parsing method
- Parser “predicts” which production to use
- It removes backtracking by fixing one production for every non- terminal and input token(s)
- Predictive parsers accept LL(k) languages

Predictive Parser

- A non recursive top down parsing method
- Parser “predicts” which production to use
- It removes backtracking by fixing one production for every non- terminal and input token(s)
- Predictive parsers accept LL(k) languages
 - ▶ First L stands for left to right scan of input

Predictive Parser

- A non recursive top down parsing method
- Parser “predicts” which production to use
- It removes backtracking by fixing one production for every non- terminal and input token(s)
- Predictive parsers accept LL(k) languages
 - ▶ First L stands for left to right scan of input
 - ▶ Second L stands for leftmost derivation

Predictive Parser

- A non recursive top down parsing method
- Parser “predicts” which production to use
- It removes backtracking by fixing one production for every non- terminal and input token(s)
- Predictive parsers accept LL(k) languages
 - ▶ First L stands for left to right scan of input
 - ▶ Second L stands for leftmost derivation
 - ▶ k stands for number of lookahead token

Predictive Parser

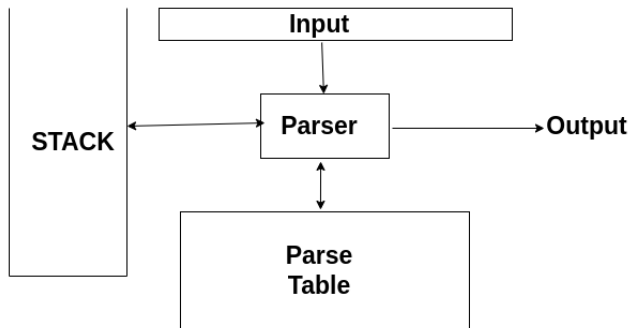
- A non recursive top down parsing method
- Parser “predicts” which production to use
- It removes backtracking by fixing one production for every non- terminal and input token(s)
- Predictive parsers accept LL(k) languages
 - ▶ First L stands for left to right scan of input
 - ▶ Second L stands for leftmost derivation
 - ▶ k stands for number of lookahead token
 - ▶ In practice LL(1) is used

Predictive Parser

Predictive parser can be implemented by maintaining an external stack

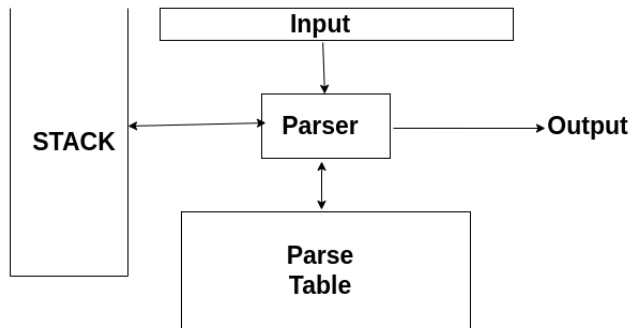
Predictive Parser

Predictive parser can be implemented by maintaining an external stack



Predictive Parser

Predictive parser can be implemented by maintaining an external stack



Parse table is a two dimensional array $M[X, a]$ where “X” is a non terminal and “a” is a terminal of the grammar

Example

Consider the following grammar:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

Parse table for the grammar

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Parse table for the grammar

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Blank entries are error states.

Parsing Algorithm

- The parser considers 'X' the symbol on top of stack, and 'a' the current input symbol.

Parsing Algorithm

- The parser considers ' X ' the symbol on top of stack, and ' a ' the current input symbol.
- These two symbols determine the action to be taken by the parser.

Parsing Algorithm

- The parser considers ' X ' the symbol on top of stack, and ' a ' the current input symbol.
- These two symbols determine the action to be taken by the parser.
- Assume that '\$' is a special token that is at the bottom of the stack and terminator for the input string

Parsing Algorithm

- The parser considers ' X ' the symbol on top of stack, and ' a ' the current input symbol.
- These two symbols determine the action to be taken by the parser.
- Assume that '\$' is a special token that is at the bottom of the stack and terminator for the input string
- if $X = a = \$$ then halt

Parsing Algorithm

- The parser considers ' X ' the symbol on top of stack, and ' a ' the current input symbol.
- These two symbols determine the action to be taken by the parser.
- Assume that '\$' is a special token that is at the bottom of the stack and terminator for the input string
- if $X = a = \$$ then halt
- if $X = a \neq \$$ then $pop(x)$ and $ip++$

Parsing Algorithm

- The parser considers ' X ' the symbol on top of stack, and ' a ' the current input symbol.
- These two symbols determine the action to be taken by the parser.
- Assume that '\$' is a special token that is at the bottom of the stack and terminator for the input string
- if $X = a = \$$ then halt
- if $X = a \neq \$$ then $pop(x)$ and $ip++$
- if X is a non terminal

Parsing Algorithm

- The parser considers ' X ' the symbol on top of stack, and ' a ' the current input symbol.
- These two symbols determine the action to be taken by the parser.
- Assume that '\$' is a special token that is at the bottom of the stack and terminator for the input string
- if $X = a = \$$ then halt
- if $X = a \neq \$$ then $pop(x)$ and $ip++$
- if X is a non terminal
then if $M[X, a] = X \rightarrow UVW$

Parsing Algorithm

- The parser considers ' X ' the symbol on top of stack, and ' a ' the current input symbol.
- These two symbols determine the action to be taken by the parser.
- Assume that '\$' is a special token that is at the bottom of the stack and terminator for the input string
- if $X = a = \$$ then halt
- if $X = a \neq \$$ then $pop(x)$ and $ip++$
- if X is a non terminal
 then if $M[X, a] = X \rightarrow UVW$
 then begin $pop(X)$; $push(W, V, U)$

Parsing Algorithm

- The parser considers ' X ' the symbol on top of stack, and ' a ' the current input symbol.
- These two symbols determine the action to be taken by the parser.
- Assume that '\$' is a special token that is at the bottom of the stack and terminator for the input string
- if $X = a = \$$ then halt
- if $X = a \neq \$$ then $pop(x)$ and $ip++$
- if X is a non terminal
 then if $M[X, a] = X \rightarrow UVW$
 then begin $pop(X)$; $push(W, V, U)$
 end

Parsing Algorithm

- The parser considers ' X ' the symbol on top of stack, and ' a ' the current input symbol.
- These two symbols determine the action to be taken by the parser.
- Assume that '\$' is a special token that is at the bottom of the stack and terminator for the input string
- if $X = a = \$$ then halt
- if $X = a \neq \$$ then $pop(x)$ and $ip++$
- if X is a non terminal
 then if $M[X, a] = X \rightarrow UVW$
 then begin $pop(X)$; $push(W, V, U)$
 end
 else error

Example

Stack	Input	Action
\$E	id+id*id \$	expand by $E \rightarrow TE'$

Example

Stack	Input	Action
\$E	id+id*id \$	expand by $E \rightarrow TE'$
\$E'T	id+id*id \$	expand by $T \rightarrow FT'$

Example

Stack	Input	Action
\$E	id+id*id \$	expand by $E \rightarrow TE'$
\$E'T	id+id*id \$	expand by $T \rightarrow FT'$
\$E'T'F	id+id*id \$	expand by $F \rightarrow id$

Example

Stack	Input	Action
\$E	id+id*id \$	expand by $E \rightarrow TE'$
\$E'T	id+id*id \$	expand by $T \rightarrow FT'$
\$E'T'F	id+id*id \$	expand by $F \rightarrow id$
\$E'T'id	id+id*id \$	pop id and ip++

Example

Stack	Input	Action
\$E	id+id*id \$	expand by $E \rightarrow TE'$
\$E'T	id+id*id \$	expand by $T \rightarrow FT'$
\$E'T'F	id+id*id \$	expand by $F \rightarrow id$
\$E'T'id	id+id*id \$	pop id and ip++
\$E'T'	+id*id \$	expand by $T \rightarrow \epsilon$

Example

Stack	Input	Action
\$E	id+id*id \$	expand by $E \rightarrow TE'$
\$E'T	id+id*id \$	expand by $T \rightarrow FT'$
\$E'T'F	id+id*id \$	expand by $F \rightarrow id$
\$E'T'id	id+id*id \$	pop id and ip++
\$E'T'	+id*id \$	expand by $T \rightarrow \epsilon$
\$E'	+id*id \$	expand by $E' \rightarrow +TE'$

Example

Stack	Input	Action
\$E	id+id*id \$	expand by $E \rightarrow TE'$
\$E'T	id+id*id \$	expand by $T \rightarrow FT'$
\$E'T'F	id+id*id \$	expand by $F \rightarrow id$
\$E'T'id	id+id*id \$	pop id and ip++
\$E'T'	+id*id \$	expand by $T \rightarrow \epsilon$
\$E'	+id*id \$	expand by $E' \rightarrow +TE'$
\$E'T+	+id*id \$	pop + and ip++

Example

Stack	Input	Action
\$E	id+id*id \$	expand by $E \rightarrow TE'$
\$E'T	id+id*id \$	expand by $T \rightarrow FT'$
\$E'T'F	id+id*id \$	expand by $F \rightarrow id$
\$E'T'id	id+id*id \$	pop id and ip++
\$E'T'	+id*id \$	expand by $T \rightarrow \epsilon$
\$E'	+id*id \$	expand by $E' \rightarrow +TE'$
\$E'T+	+id*id \$	pop + and ip++
\$E'T	id*id \$	expand by $T \rightarrow FT'$

Example

Stack	Input	Action
\$E'T'F	id*id \$	expand by $F \rightarrow id$

Example

Stack	Input	Action
\$E'T'F	id*id \$	expand by $F \rightarrow id$
\$E'T'id	id*id \$	pop id and $ip++$

Example

Stack	Input	Action
\$E'T'F	id*id \$	expand by $F \rightarrow id$
\$E'T'id	id*id \$	pop id and $ip++$
\$E'T'	*id \$	expand by $T' \rightarrow *FT'$

Example

Stack	Input	Action
\$E'T'F	id*id \$	expand by $F \rightarrow id$
\$E'T'id	id*id \$	pop id and $ip++$
\$E'T'	*id \$	expand by $T' \rightarrow *FT'$
\$E'T'F*	*id \$	pop * and $ip++$

Example

Stack	Input	Action
\$E'T'F	id*id \$	expand by $F \rightarrow id$
\$E'T'id	id*id \$	pop id and $ip++$
\$E'T'	*id \$	expand by $T' \rightarrow *FT'$
\$E'T'F*	*id \$	pop * and $ip++$
\$E'T'F	id \$	expand by $F \rightarrow id$

Example

Stack	Input	Action
\$E'T'F	id*id \$	expand by $F \rightarrow id$
\$E'T'id	id*id \$	pop id and ip++
\$E'T'	*id \$	expand by $T' \rightarrow *FT'$
\$E'T'F*	*id \$	pop * and ip++
\$E'T'F	id \$	expand by $F \rightarrow id$
\$E'T'id	id \$	pop id and ip++

Example

Stack	Input	Action
\$E'T'F	id*id \$	expand by $F \rightarrow id$
\$E'T'id	id*id \$	pop id and $ip++$
\$E'T'	*id \$	expand by $T' \rightarrow *FT'$
\$E'T'F*	*id \$	pop * and $ip++$
\$E'T'F	id \$	expand by $F \rightarrow id$
\$E'T'id	id \$	pop id and $ip++$
\$E'T'	\$	expand by $T' \rightarrow \epsilon$

Example

Stack	Input	Action
\$E'T'F	id*id \$	expand by $F \rightarrow id$
\$E'T'id	id*id \$	pop id and $ip++$
\$E'T'	*id \$	expand by $T' \rightarrow *FT'$
\$E'T'F*	*id \$	pop * and $ip++$
\$E'T'F	id \$	expand by $F \rightarrow id$
\$E'T'id	id \$	pop id and $ip++$
\$E'T'	\$	expand by $T' \rightarrow \epsilon$
\$E'	\$	expand by $E' \rightarrow \epsilon$

Example

Stack	Input	Action
\$E'T'F	id*id \$	expand by $F \rightarrow id$
\$E'T'id	id*id \$	pop id and $ip++$
\$E'T'	*id \$	expand by $T' \rightarrow *FT'$
\$E'T'F*	*id \$	pop * and $ip++$
\$E'T'F	id \$	expand by $F \rightarrow id$
\$E'T'id	id \$	pop id and $ip++$
\$E'T'	\$	expand by $T' \rightarrow \epsilon$
\$E'	\$	expand by $E' \rightarrow \epsilon$
\$	\$	halt